

Collin Backus
CS3110-001
Sprint Reflection 3
Due: April 4, 2026

Goals

1. *Establish Posting Architecture*

Met (y/n): y

Posting architecture came after user posting. Originally, a post was a simple plain text line with the username that posted in parentheses following the text of the post. This was done to facilitate simple testing of user posting. Once user posting was confirmed as working, the plain text post was upgraded to a function in the front end JS function. This function dynamically builds an HTML framework that is populated with the requisite data for a post: the profile picture, username, and the post text. I made this a function after I had written the functionality solely in loadPosts for the index page, only to realize that I would duplicate the same functionality in the loadProfile function too. It was fairly straightforward to just make it a standalone function and call it from the two places that use it.

2. *Establish User Posting*

Met (y/n): y

This was done by using a form submission event that reads the data from the post-input element and gets the user's current location, then sends this data to the backend JS file. This file checks if a user is logged in and if a location has been captured, then assembles the post data as an object and pushes this object into the array of posts in memory. This post contains the id of it in the array, the username associated with the post, the text content, the location of the user when it was posted, and the date it was posted. This data is used to parse the array for the profile and index pages to show either data that is associated with the username or the location radius, respectively.

3. *Index page must show all user posts within specific geographic radius*

a. Radius determined by geolocation services

Met (y/n): y

Originally, all posts were retrieved from memory and loaded on the index page. This was upgraded to filter based on location by creating a front end JS function that retrieved the geolocation of the logged in user in the form of latitude and longitude coordinates and attached to every post a user makes. The coordinates of the user and the coordinates of the posts are compared in the backend when fetching the posts from memory on the basis of distance in miles. I used the Haversine Formula to approximate the distance between two latitude-longitude coordinates in miles and the get API returns all posts that are within the specified radius in miles. In this case, five miles. The functionality of this was verified by

both myself logging in as user2 and traveling more than five miles away from my home and looking for a post made by user1 while at home, and by having a friend open the website and login as well.

4. *Show user posts on associated user profile page*

Met (y/n): y

This was simply implemented by fetching all posts from memory using the `assemblePosts` function, described in Goal 1, that are posted by the specific username of the profile page that is associated with posts in memory.

5. *Visit other users' profile pages*

a. *See the associated user's posts*

Met (y/n): y

Visiting another user's profile page was originally implemented by typing the raw HTML page address with query parameters, i.e., `/profile.html?username=user2`. This was upgraded to allow this navigation by clicking on the username as it appears in the index feed by using a click event and changing the cursor from the standard cursor image used for text to a pointer type image. Seeing the associated user's posts was done by implementing the same function that allows a user to see their own posts.

6. *User profile page must reflect user's username*

Met (y/n): y

Originally its own function, this was brought into the `loadProfile` function in the front end JS file as three simple if cases based on `getElementById` for the three places the username is found on the profile page. These used to be static "@username" placeholders in the page title, the black header bar, and the main body of the page. These if statements are dynamic and work for the current logged in user visiting their own page and visiting another user's profile page.

7. *Users must allow geolocation services*

a. *Accept terms when registering account*

b. *This allows visible posts to be filtered in the backend to a specific geographic radius*

Met (y/n): y

Upon registering an account, the user must authorize location services to be used via a popup from their browser. Geolocation services were implemented in the front end JS file by calling the "navigator.geolocation" API. This returns the latitude and longitude coordinates of the device of a user, which is then sent to the backend JS file that compares the user's coordinates to a post's coordinates and finds the distance between them to determine whether or not the post is shown to the user.

Next Sprint

The next sprint will focus on post interaction architecture. The interactions for a post are “likes,” “reblogs,” and “replies.” All interactions will be shown with a counter next to their icon in the footer of the post architecture. Reblogs will be shared on the reblogger’s profile and replies will be shown as children of the original post. With the heavy lifting of user posting done, I’m hoping that this next sprint will be a bit less intense.

Link to Deployed Application and Usage/Testing Instructions

<https://myconet.chickenkiller.com/>

Posts made on the index page are visible there and are also visible on the profile page associated with the user that made the post. Multiple users are supported. Current active credentials are below. Because I’ve had to restart the server a few times and I am not yet storing user/post data persistently, you must register again or use the credentials below. You may log out by clicking the logout button on either the index page or the profile.

Email	Password	Username
backus.collin@yahoo.com	test	user1
collinbackus@gmail.com	test	user2